

---

# MEMORY PERFORMANCE MODELING AND PARALLEL MEMORY SYSTEMS

# Memory Performance Modeling

---

## Topics:

- The Memory Wall
- Latency vs Bandwidth
- Performance Modeling
- Roofline Model
- Memory-Level Parallelism (MLP)
- Non-Blocking Caches and MSHRs
- Prefetching
- NUMA Architectures
- Multicore Memory Scaling
- Real-World
- Case Studies

# Motivation

---

So far we studied cache hierarchies, coherence, consistency, virtual memory and DRAM

However, understanding individual components does not automatically explain system performance.

This lesson focuses on **System-Level Behavior**

How memory subsystems affect:

- execution time
- scalability
- throughput
- energy consumption

# Memory Wall Recap

Since the 1980s processor performance improved much faster than memory latency.

| Year  | CPU performance | DRAM latency       |
|-------|-----------------|--------------------|
| 1980  | baseline        | Baseline           |
| 2000  | x1000           | X10                |
| Today | Much higher     | Modest improvement |

Processors spend increasing amounts of time waiting for memory.

This phenomenon is called the Memory Wall

# Why the Memory Wall Exists

---

Improvements in CPUs come from:

- deeper pipelines
- superscalar execution
- out-of-order execution
- higher frequencies

DRAM improvements are limited by:

- capacitor physics
- signal integrity
- power constraints

# CPU Performance Equation

---

Recall:

$$CPU\ time = IC \times CPI \times Clock\ Cycle$$

Where  $IC$  is the instruction count and  $CPI$  are the cycles per instruction

In a real system the CPI includes also the memory contribution

$$CPI = CPI_{base} + Memory\ Stall\ Cycles$$

# Example

---

Suppose  $CPI_{base} = 1$  and *Memory Stall Cycles* = 4

The total CPI is 5

Interpretation:

- only 20% of execution is useful work
- the remaining 80% is waiting for memory

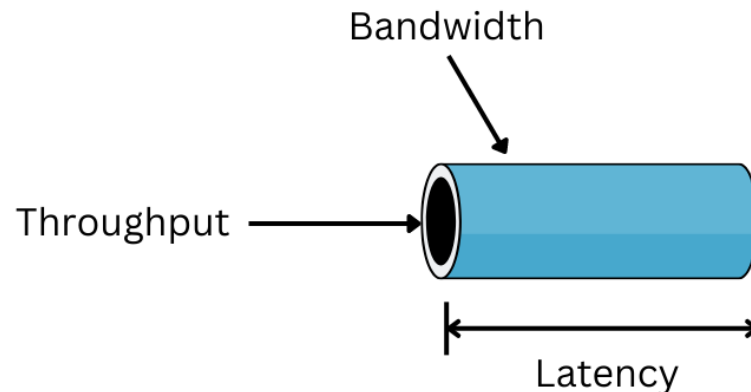
# Latency vs Bandwidth

Latency is the time required to complete one request

example: latency of 100 ns

Bandwidth is the amount of data transferred per second

example: bandwidth of 50 GB/s





# Why Both Matter

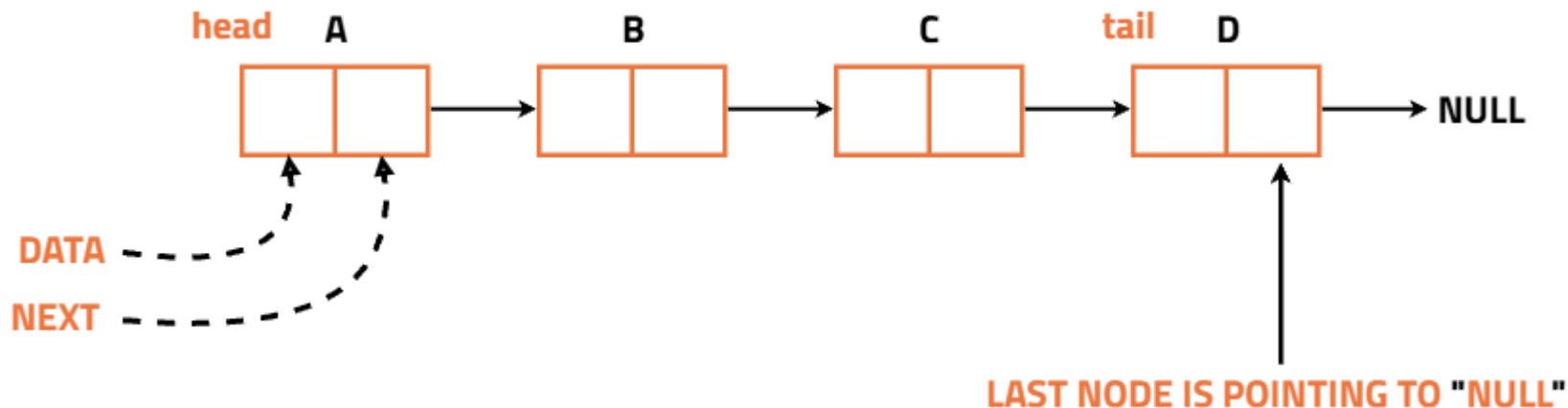
Example: linked list traversal

```
p = head;
```

```
while(p)
```

```
    p = p->next;
```

Performance depends mainly on latency

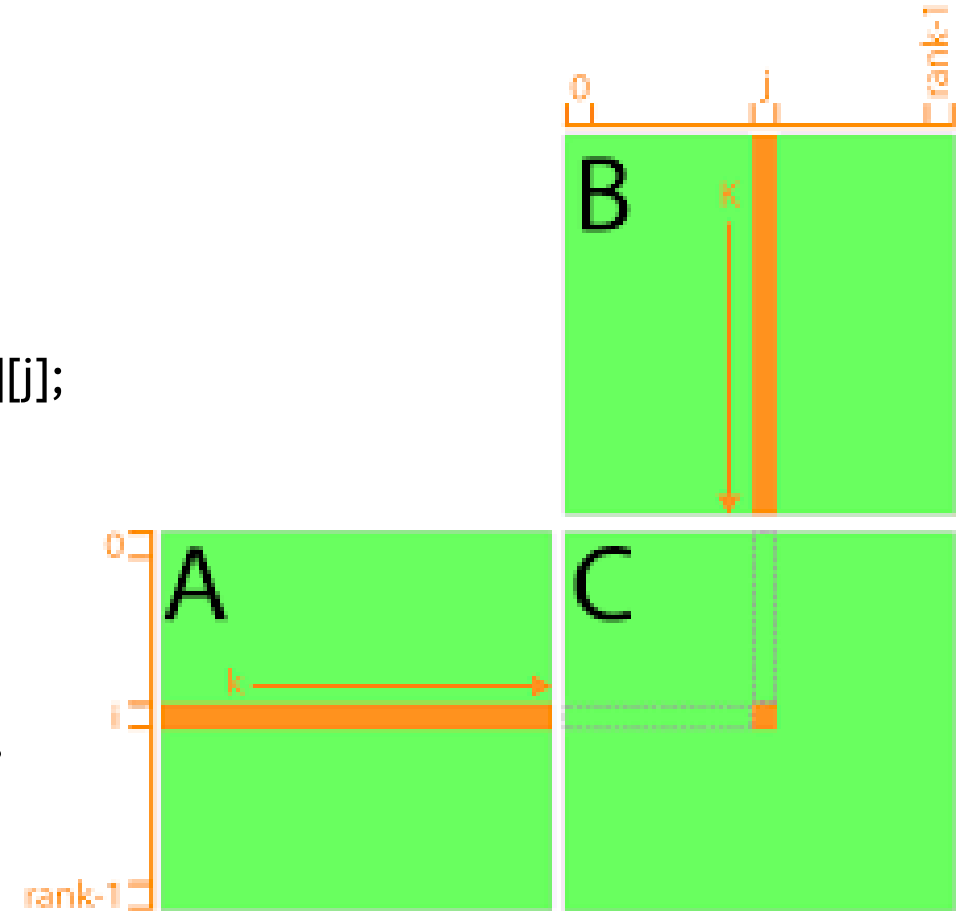


# Why Both Matter

## Example: matrix multiplication

```
for (int i = 0; i < R1; i++) {  
    for (int j = 0; j < C2; j++) {  
        result[i][j] = 0;  
        for (int k = 0; k < R2; k++) {  
            result[i][j] += m1[i][k] * m2[k][j];  
        }  
    }  
}
```

Performance depends mainly on bandwidth.



# Memory Bandwidth

---

$$\textit{Bandwidth} = \textit{BusWidth} \times \textit{Transfer Rate}$$

Example: consider a memory with a 64-b bus and a transfer rate of 6400 MT/s

The Bus Width is 64-b = 8B

The bandwidth is  $8 \times 6.4 \times 10^9 = 51.2$  GB/s

# Memory Bandwidth

---

Modern processors may have multiple channels:

- 4 channels
- 6 channels
- 8 channels
- 12 channels

Thus, if the memory of the previous example has 8 channels, the bandwidth is

$$8 \times 51.2 = 409.6 \text{ GB/s}$$

# Compute-Bound vs Memory-Bound

---

Compute-Bound means that the processor limits performance

Examples:

- Dense matrix multiplication
- Tensor operations

Memory-Bound means that the memory systems limits performance

Examples:

- Graph processing
- Database scans
- Sparse matrices

# Introduction to the Roofline Model

---

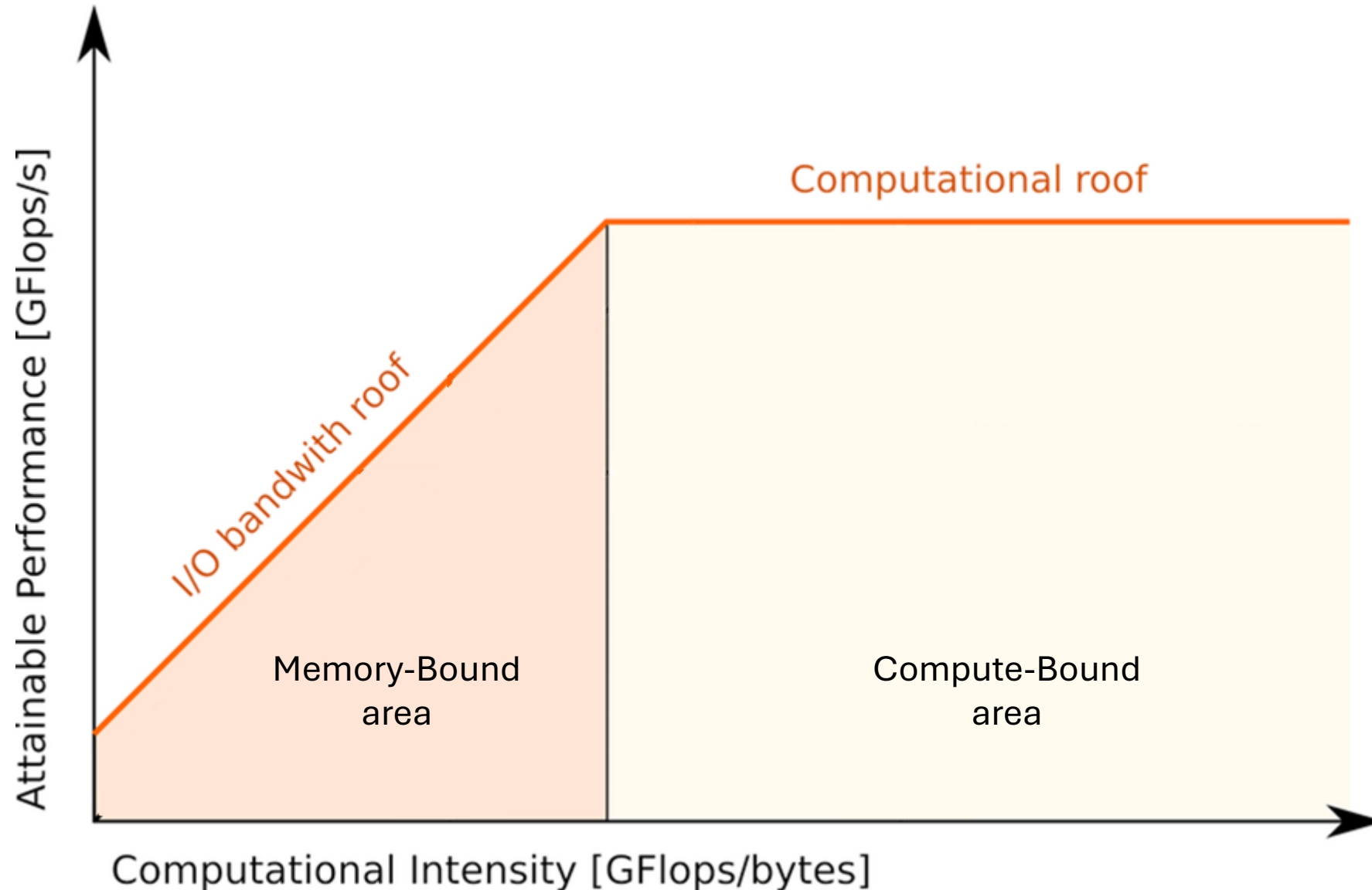
The Roofline model provides a visual performance bound

It combines:

- Peak compute performance
- Peak memory bandwidth

Into one graph

# Introduction to the Roofline Model



# Computational Intensity

---

Definition:

$$CI = \frac{\textit{Operations}}{\textit{Bytes transferred}}$$

Units of measure: FLOP/Bytes

Low CI -> many memory accesses and few computations

High CI -> many computations and relatively few memory accesses



# Roofline Equation

---

Performance is bounded by:

$$Performance = \min(PeakCompute, CI \times Bandwidth)$$

Example: consider a simple addition

$$C = A + B$$

Operations: 1 addition

Data: 3 values

Very low CI, thus memory bound

# Roofline Equation

---

Consider now the matrix-matrix multiplication

$$C = A \times B \text{ (triple for loop)}$$

Data reused many times!

High CI, thus compute bound

# Memory Level Parallelism

---

Definition: the ability to overlap multiple memory accesses.

or

The processor's ability to handle multiple pending memory requests (like cache or TLB misses) at the same time.

Instead of:

Miss 1 -> Wait -> Miss 2 -> Wait -> Miss 3 -> Wait

We have:

Miss 1 -> Miss 2 -> Miss 3  
simultaneously

# Memory Level Parallelism

---

Suppose that a DRAM had a latency of 100 ns

Without memory level parallelism 10 accesses take 1000 ns

With perfect memory level parallelism 10 accesses take 100 ns

Sources of Memory Level Parallelism:

- Out-of-order Execution
  - Independent instructions continue execution
- Non-Blocking Caches
  - Allow multiple outstanding misses
- Bank-Level parallelism
  - Different DRAM banks operate simultaneously

# Non-blocking cache

---

Traditional caches stop when a miss occurs until it is resolved

Non-blocking caches continue executing and issues more misses

The Miss Status Holding Registers (MSHRs) track outstanding misses.

Store:

- Missing block address
- Waiting instructions
- State information

# Limitation of Memory Level Parallelism

Consider  $p = p \rightarrow \text{next}$ ;

Each access depends on previous access,

This situation is known as pointer chasing. Very little memory level parallelism possible.

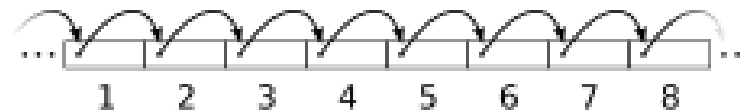


# Memory Access Patterns

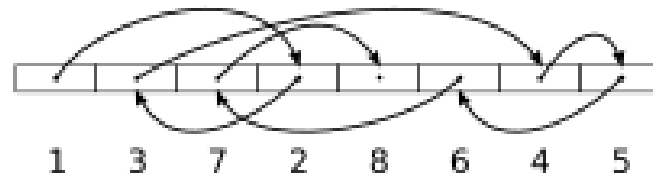
Three important patterns to access the memory:

- Sequential -> excellent locality  
0,1,2,3,4
- Strided -> optimal for some DMAs and linear algebra  
0,4,8,12
- Random

Sequential access



Random access



# Hardware Prefetching

---

Goal: predict future accesses

Load data before it is requested

Benefits: hide latency

Different strategy for hardware prefetching:

- Next-line prefetcher
- Stride prefetcher



# Hardware Prefetching

---

Next-Line Prefetcher:

Access: block  $i$   $\rightarrow$  automatically fetch: block  $i+1$

A very simple strategy that works well for sequential streams

Stride Prefetcher:

Detects: 0, 4, 8, 12, 16  $\rightarrow$  stride of +4  $\rightarrow$  predicts next accesses

Useful for scientific workloads

# Risks of Hardware Prefetching

---

Incorrect predictions:

- Waste bandwidth
- Pollute caches
- Increase energy

Good prefetchers balance:

- Coverage
- accuracy

# Non-Uniform Memory Access

---

Modern servers have:

- Multiple CPUs
- Multiple memory controllers
- Local DRAM

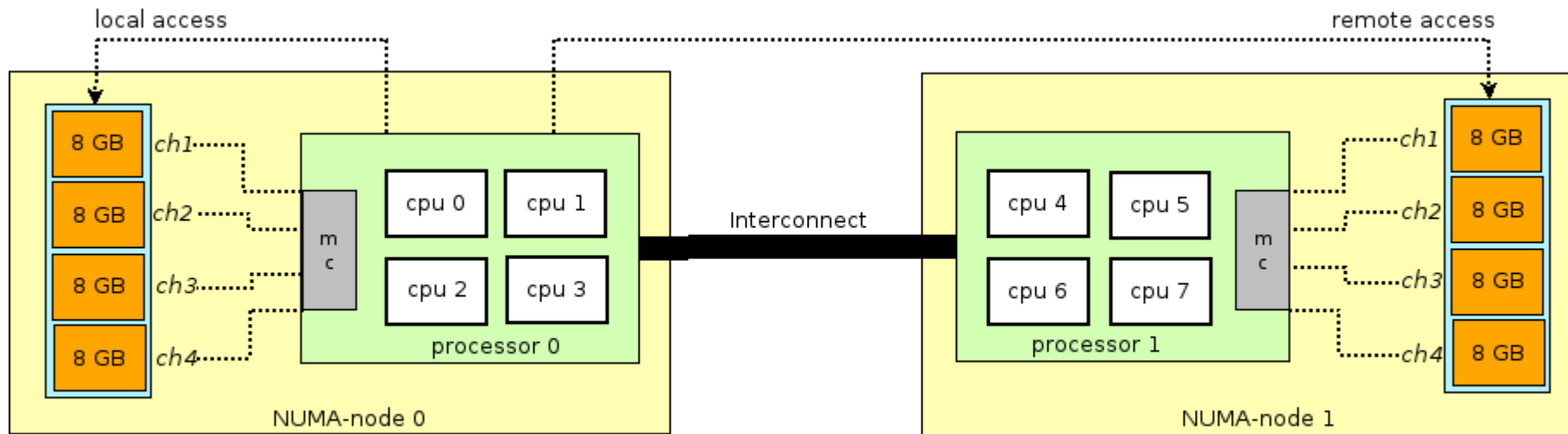
Memory latency depends on location

Example: CPU 0 has a local DRAM with 80 ns of latency and a remote DRAM of 140 ns

Remote accesses can be nearly twice as expensive

# First-Touch Policy

Operating systems often allocate memory to the NUMA node of the thread that first accesses it.

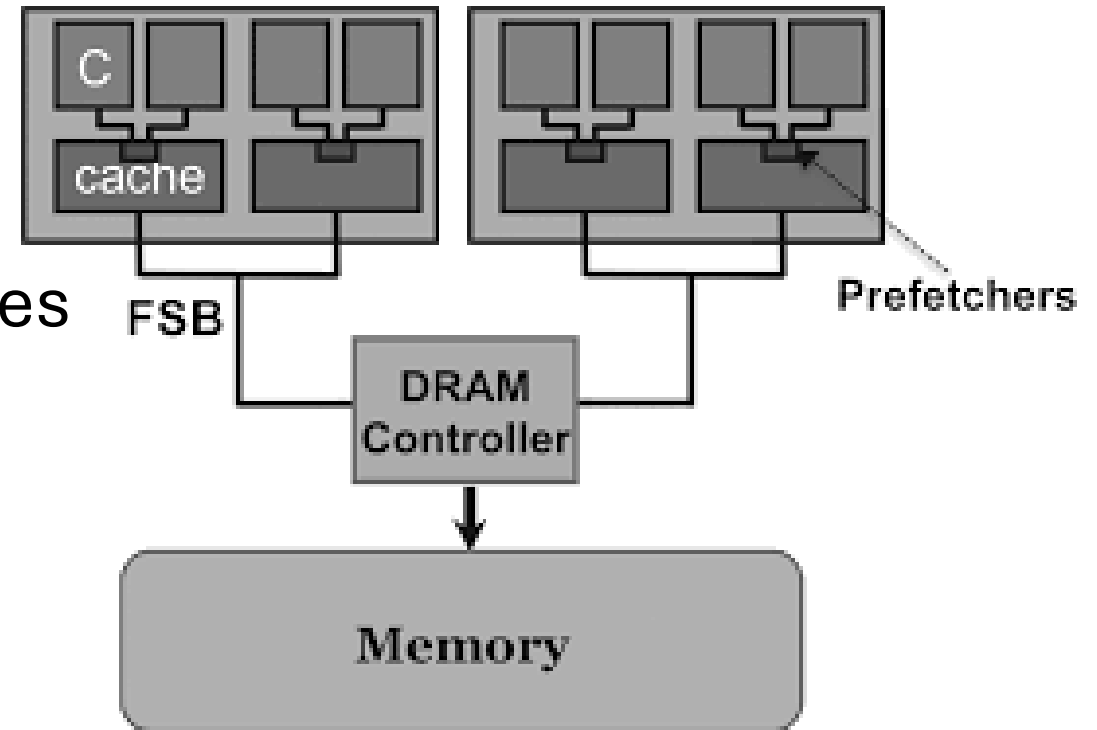


# Multicore Memory Contention

All core share:

- Memory channels
- Memory controllers
- DRAM bandwidth

As core increase the contention increases

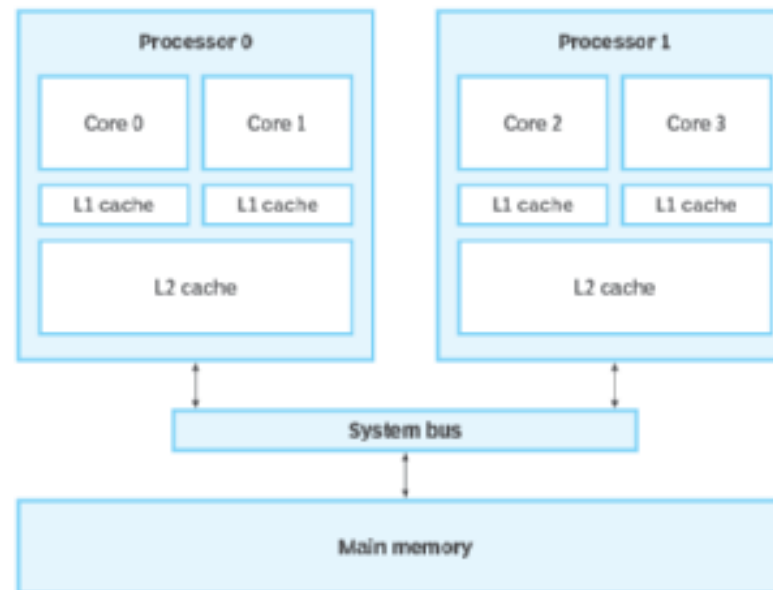


# Bandwidth per Core

Example: 64-core processor with a total bandwidth of 256 GB/s

The bandwidth per core is 4 GB/s

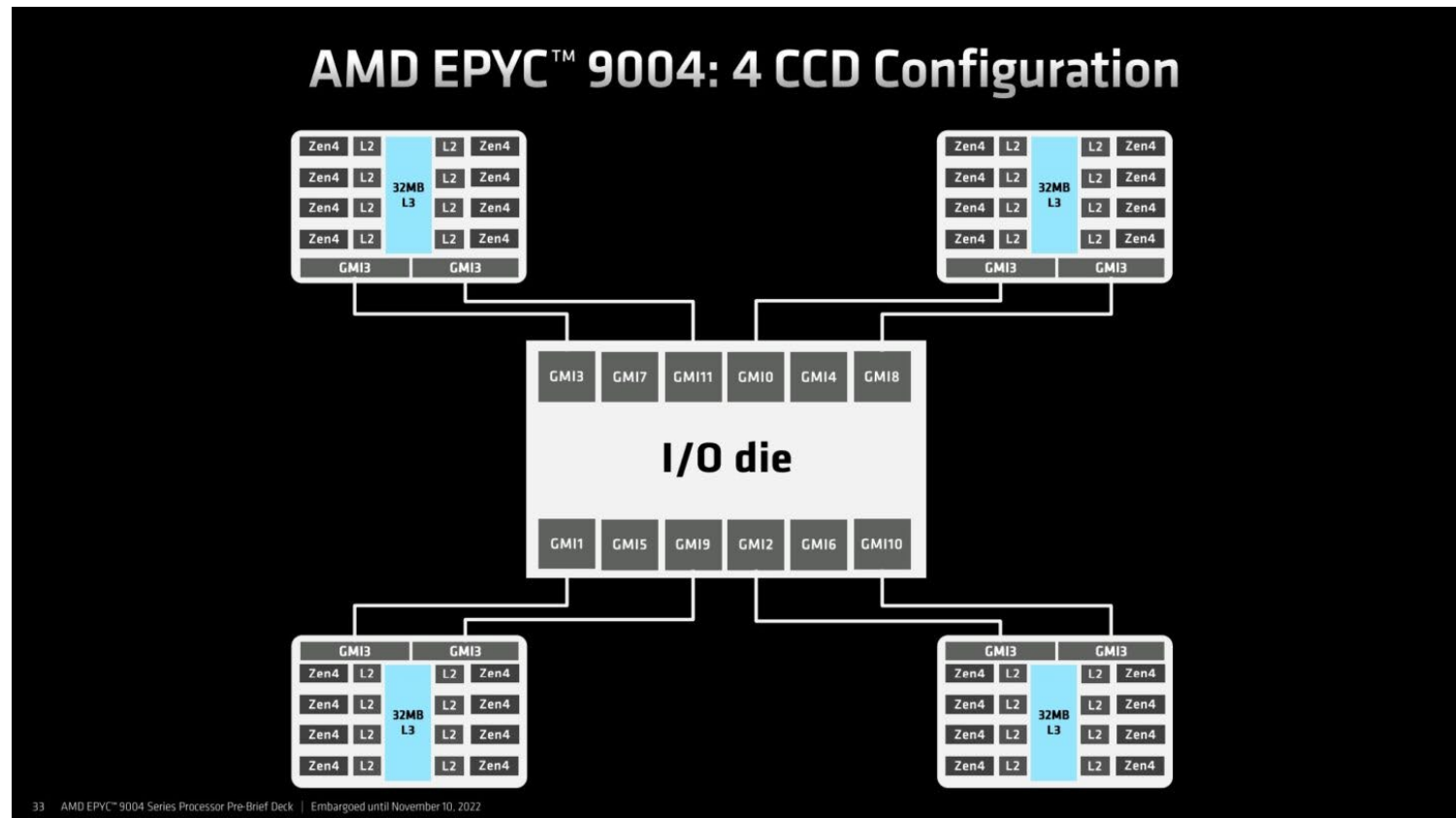
Adding cores does not automatically increase bandwidth



# Case Study: AMD EPYC

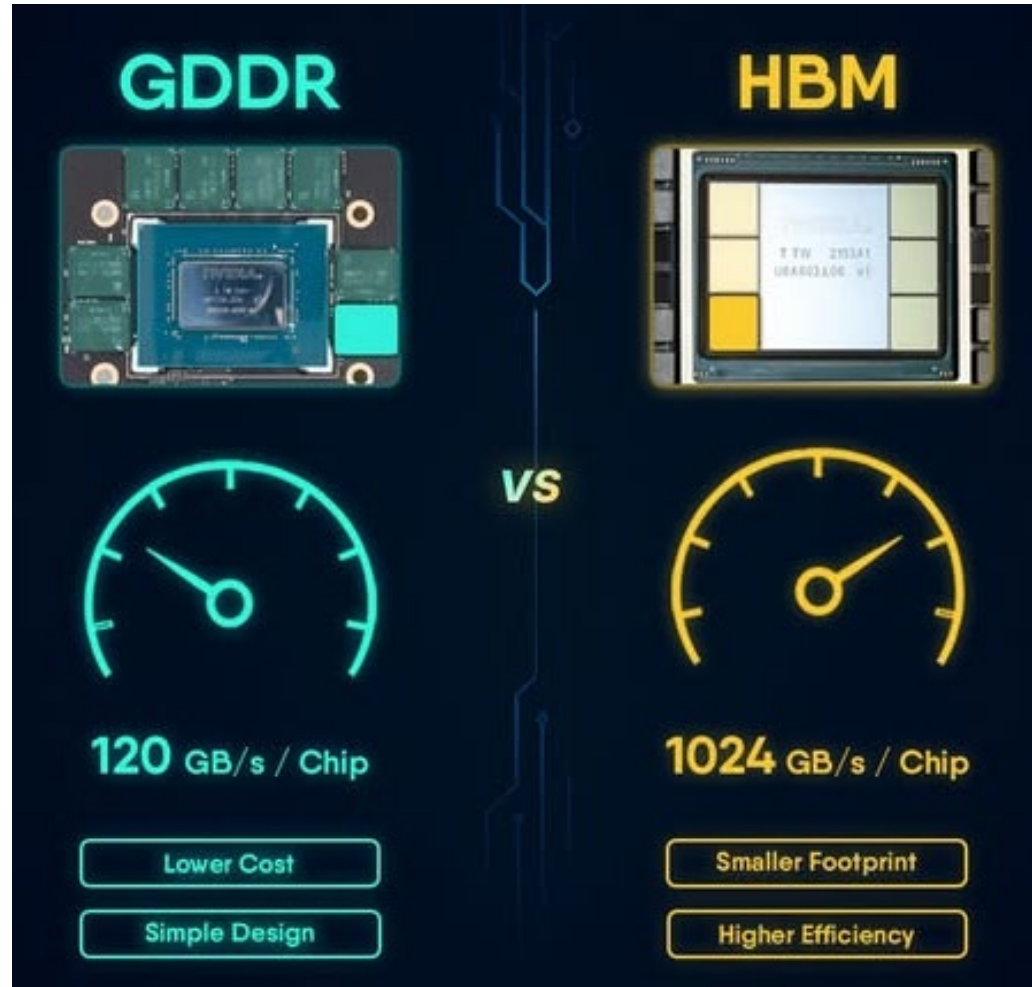
Many cores and many memory channels

Goal: increase memory bandwidth scalability



# Case Study: AI Accelerators

GPUs use HBM memory, thus the bandwidth is often  $> 1\text{TB/s}$



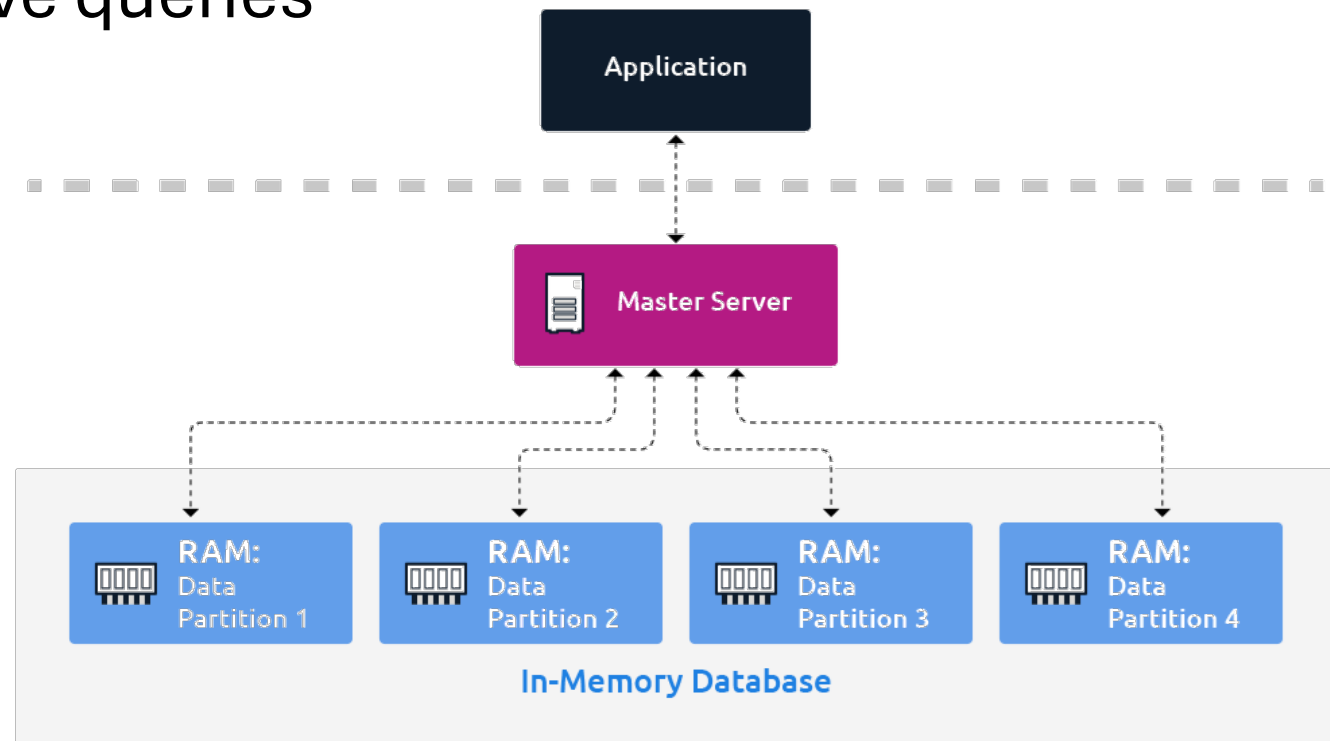


# Case Study: Databases

## Characteristics:

- Huge datasets
- Random accesses
- Memory –intensive queries

Often memory-bound.



# Design Guidelines

---

To improve performance:

1. Increase locality
2. Increase computational intensity
3. Exploit memory level parallelism
4. Reduce memory traffic

Hardware and software must co-operate.

# Closing

---

Key messages:

- Memory wall remains a major challenge
- Latency and bandwidth are different
- Roofline explains performance limits
- MLP hides latency
- NUMA affects scalability
- Modern systems are often memory-bound